



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

AI-BASED CONSTRAIN SATISFACTION SOLVER FOR AUTOMATED TIMETABLE SCHEDULING

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1709-Artificial Intelligence

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by
KAVIYA.G(192524281)

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Under the Supervision of

Dr. P RAJARAM

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “AI-BASED CONSTRAIN SATISFACTION SOLVER FOR AUTOMATED TIMETABLE SCHEDULING has been carried out by KAVIYA.G(192524281) under the supervision of Dr.P. Rajaram and is submitted in partial fulfilment of the requirements for the current semester of the program B.TECH ARTIFICIAL INTELIGENCE AND DATA SCIENCE at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

DR. KRISHNA KUMAR KATHIRESAN
Professor
CSE/Generative AI
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr Rajaram P
Professor
CSE/Data Vista,
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, KAVIYA.G of B.TECH ARTIFICIAL INTELLIGENCE AND DATA SCIENCE the SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled AI-BASED CONSTRAIN SATISFACTION SOLVER FOR AUTOMATED TIMETABLE SCHEDULING is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Chennai

Date:05/09/2026

Signature of the Students with Names

Kaviya.G(192524281)

Individual Contribution Statement

Student Name / Reg. No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Contribution (%)
T. S. Vidhushini 192511061	Module 1 – Constraint modelling and data-collection interface	Data Management UI (Add Subject/Faculty/Room), input validation	Unit testing of data-entry forms and constraint model build-up	Ch.1, Ch.3 (3.1–3.3)	34%
K. S. Dharshana 192412015	Module 3 – Timetable generation and conflict-reporting interface	Dashboard, generated-timetable view, constraint status panel	End-to-end verification of generated timetables against constraints	Ch.3 (3.4–3.7), Ch.4	33%
G. Kaviya 192524281	Module 2 – CSP solving engine (backtracking / local search)	Backtracking solver, MRV heuristic, forward checking logic	Performance testing (generation time, optimality, conflict count)	Ch.2, Ch.5	33%

Total: 100%

Abstract

Building a timetable for an engineering department is a textbook case of a combinatorially explosive scheduling challenge: each subject session has to be assigned a specific day, period and room in such a way that no faculty member, room or student section ends up double-booked, while every subject still gets its full quota of weekly periods within a fixed slot grid. Done by hand, this is slow, prone to mistakes, and often produces clashes that only come to light after the timetable has already been published — causing disruption for both staff and students. This project tackles the engineering challenge of automatically producing a conflict-free, optimal class timetable, framed here as a Constraint Satisfaction Problem (CSP). The solution proposed here, the AI Scheduler, is a web-based system built around three modules: (i) a data-collection module that gathers subject information, faculty availability, room capacity/type and the institution's time-slot grid; (ii) a CSP solving engine that treats faculty-conflict, room-conflict, room-capacity, time-availability, weekly-hours and section-conflict rules as hard constraints, and explores the assignment space through backtracking search guided by a minimum-remaining-values (MRV) heuristic and forward checking, with room to add local-search refinement (simulated annealing) once the search space expands; and (iii) a timetable-generation and conflict-reporting interface that displays the resulting schedule along with the status of each constraint. Development followed a build-test-verify cycle: each constraint was encoded and checked on its own before the solver was tested against a representative department-scale dataset — four subjects, four faculty members and three rooms spread across a 5-day, 5-period (25-slot) grid. The system reliably produced an OPTIMAL solution, placing all twelve required class sessions in 0.357 seconds with no constraint violations across any of the six active constraint categories. These results show that a heuristic backtracking approach to CSP is a good fit for department-scale academic timetabling, offering both correctness (zero conflicts) and practicality (sub-second generation) without needing specialised optimisation infrastructure. The main takeaway is that automated, constraint-driven scheduling can dependably take the place of manual timetable preparation at this scale, all while staying transparent and auditable to faculty — and that the underlying architecture can be extended to larger, multi-department deployments through local-search optimisation and persistent data storage.

Table of Contents

Sl. No.	Title	Page No.
i	Certificate from the Supervisor	ii
ii	Declaration by the Candidate	iii
iii	Individual Contribution Statement	iv
iv	Abstract	v
v	List of Figures	vi
vi	List of Tables	vii
vii	List of Abbreviations	viii
1	Chapter 1: Introduction and Engineering Problem	—
2	Chapter 2: Literature Review and Existing Solutions	—
3	Chapter 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	—
4	Chapter 4: Implementation, Testing & Results, Project Management	—
5	Chapter 5: Conclusion, Future Work and Reflection	—
	References	—
	Appendices	—
	Capstone Project Outcome Mapping Sheet	—

List of Figures

Figure No.	Figure Name	Page No.
Fig. 3.1	System Architecture / Module Flow Diagram	—
Fig. 4.1	AI Scheduler – Data Management Interface	—
Fig. 4.2	Scheduling Constraints Dashboard (Active Rule Set)	—
Fig. 4.3	AI-Generated Timetable Output (Sample Rows)	—

List of Tables

Table No.	Table Name	Page No.
Table 2.1	Comparative Analysis of Existing Timetabling Approaches	—
Table 3.1	Stakeholder / User Requirements	—
Table 3.2	Functional and Non-Functional Requirements	—
Table 3.3	Applicable Constraints and Standards	—
Table 3.4	Design Specifications	—
Table 3.5	Evaluation of Alternative Solutions	—
Table 3.6	Trade-off Analysis	—
Table 3.7	Sustainability, Ethics, Safety and Security Considerations	—
Table 3.8	Risk Register	—
Table 4.1	Tools, Software and Equipment Used	—
Table 4.2	Test Plan and Verification	—
Table 4.3	Achieved Results vs. Required Values	—
Table 4.4	Achievement of Objectives	—
Table 4.5	Project Milestones	—
Table 4.6	Roles and Contribution	—
Table 4.7	Estimated vs. Actual Cost	—

List of Abbreviations / Symbols

Symbol	Description	Unit
AI	Artificial Intelligence	—
CSP	Constraint Satisfaction Problem	—
MRV	Minimum Remaining Values (heuristic)	—
GA	Genetic Algorithm	—
ILP	Integer Linear Programming	—
UI	User Interface	—
DB	Database	—
LMS	Learning Management System	—
s	Seconds (generation time)	sec

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Academic timetabling means fitting a fixed set of teaching sessions into a limited grid of days, periods and rooms so that every subject gets its required contact hours while no faculty member, room or student section ends up double-booked. In a department such as Electronics and Communication Engineering at SIMATS, this means balancing subject-specific weekly-hour requirements, faculty availability, room capacity/type (classroom versus laboratory) and a fixed institutional slot grid (typically five working days of five periods each, i.e. 25 slots a week). The underlying assignment task is a well-recognised instance of the Constraint Satisfaction Problem (CSP), and in its general form it is NP-hard: the number of possible day-time-room combinations grows combinatorially with the number of subjects, faculty and rooms, which makes exhaustive manual search impractical once there are more than a handful of subjects.

1.2 Need for the Project

Timetables prepared by hand at department scale typically take several days of coordinator effort and still end up with residual clashes — a faculty member booked for two sections at the same time, a laboratory scheduled beyond its capacity, or a subject that falls short of its weekly-hour requirement. These problems tend to surface late, often after the timetable has already gone out, forcing repeated manual fixes. The people who feel this most directly are class coordinators (who carry the preparation load), faculty (who face last-minute reassignment), and students (whose sections may be double-booked or left with an uneven number of contact hours). Current departmental practice relies on spreadsheet-based, trial-and-error allocation with no systematic way of checking for conflicts, and this approach does not scale as the number of subjects, sections and faculty increases. There is, then, a clear engineering case for a system that encodes scheduling rules explicitly and automatically searches for an assignment that is provably conflict-free.

1.3 Problem Statement

Given a set of subjects, each needing a fixed number of periods per week and taught by a named faculty member to a defined student section of known size; a set of rooms, each with a fixed capacity and type (classroom or laboratory); and a fixed weekly slot grid of five days and five periods (25 slots), the task is to assign every required subject session to a (day, period, room) combination such that:

- a faculty member is never assigned to two sessions in the same slot (faculty-conflict constraint);
- a room is never assigned to two sessions in the same slot (room-conflict constraint);
- the section's student strength never exceeds the capacity of the room it is assigned to (room-capacity constraint);
- sessions are only placed in slots the relevant faculty member has marked as available (time-availability constraint);
- each subject is given exactly its required number of weekly periods, no more and no less (weekly-hours constraint); and
- a student section is never assigned to two sessions in the same slot (section-conflict constraint).

This is an engineering-relevant problem because the six constraints interact with, and can work against, one another within a fixed slot budget — for instance, meeting every subject's weekly-hour requirement while respecting limited room and faculty availability is not always straightforwardly achievable, so the solver has to either find a feasible assignment or clearly identify which constraint(s) are blocking one.

1.4 Project Aim

To design and build an AI-based Constraint Satisfaction Problem solver capable of automatically generating a conflict-free, optimised class timetable for an engineering department — replacing manual, error-prone scheduling with a transparent, rule-driven, web-based system.

1.5 Project Objectives

1. To formally model the constraints covering faculty, room, capacity, availability, weekly-hours and section-conflict rules.
2. To build a data-collection interface for entering subjects, faculty, rooms and the time-slot grid.
3. To implement a backtracking CSP engine that uses heuristic variable ordering (MRV) together with forward checking.
4. To build in local-search-based refinement (simulated annealing) so the system can scale beyond small, fully-constrained instances.
5. To implement automatic conflict detection alongside a constraint-status reporting dashboard.
6. To test the solver against a representative department-scale dataset and assess its generation time, conflict count and solution optimality.

1.6 Scope

In scope: intra-department timetabling for a single institution, covering the faculty-conflict, room-conflict, room-capacity, time-availability, weekly-hours and section-conflict constraints within a fixed 25-slot weekly grid, delivered through a browser-based dashboard (Data Management, Constraints, Generate, Reports).

Out of scope: inter-department elective clashes, multi-campus or multi-shift scheduling, real-time dynamic rescheduling in response to ad hoc leave or cancellation, and integration with an institutional Learning Management System.

Assumptions made: the weekly slot grid stays fixed at five days and five periods; subject, faculty and room data are entered once per generation cycle and held static while solving; each subject session takes up exactly one slot.

Boundary conditions: the solver has been evaluated on department-scale inputs (on the order of tens of subjects, faculty and rooms); very large, multi-department instances are flagged in Section 5.2 as future work that would need additional optimisation techniques.

1.7 Expected Engineering Outcome

The intended outcome is a working software prototype, the AI Scheduler, made up of a data-management module, a CSP-based solving engine and a timetable-generation/reporting interface, able to produce an optimal, conflict-free weekly timetable for a representative department dataset within a few seconds — along with a constraint-satisfaction status report confirming that all active scheduling rules have been met.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing tools were located through a mix of foundational computer-science references on the complexity of timetabling, established constraint-satisfaction and AI textbooks, open-source constraint-programming libraries (python-constraint, Google OR-Tools), and widely used academic or commercial timetabling software (e.g., FET — Free Timetabling Software, aSc TimeTables). The review concentrated on how each approach models scheduling constraints and searches through the assignment space.

2.2 Review of Existing Work

Timetabling is formally recognised as a computationally hard scheduling problem, closely related to graph-colouring and multi-commodity flow formulations, which explains why exhaustive manual or brute-force methods fail to scale [1]. Classical AI treatments of Constraint Satisfaction Problems present backtracking search combined with heuristics such as minimum-remaining-values (MRV) variable ordering and forward checking/arc-consistency as effective general-purpose techniques for problems of this kind [2], and it is this body of theory that underpins the solving engine built for this project.

Beyond pure backtracking, the broader literature on automated timetabling covers metaheuristic approaches — genetic algorithms, simulated annealing and tabu search — which give up solving-time guarantees in exchange for the ability to handle much larger, softly-constrained problem instances, accepting near-optimal rather than strictly optimal solutions [6]. Open-source constraint solvers such as Google's OR-Tools offer production-grade CP-SAT solvers that can handle both the hard constraints modelled here and additional soft objectives at larger scale [3], while lightweight libraries such as python-constraint provide a simpler backtracking API that suits rapid prototyping of department-scale problems well [4]. Existing commercial and open-source timetabling products (e.g., FET, aSc TimeTables) implement broadly similar constraint sets, but are usually delivered as standalone desktop applications rather than the lightweight, browser-accessible dashboards that non-technical faculty coordinators could operate directly.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual / spreadsheet scheduling	No tooling cost; full human control	Slow, error-prone, no systematic conflict check, does not scale	Motivates the need for an automated CSP-based solver
Pure backtracking CSP (general)	Guarantees a correct, conflict-free solution if one exists; simple to model	Can be slow on large, loosely-constrained instances without good heuristics	Adopted as the core solving strategy, combined with MRV + forward checking
Genetic Algorithm / metaheuristics	Scales to very large, softly-constrained problems; good near-optimal solutions	No optimality guarantee; requires tuning of population/mutation parameters	Identified as a future extension for larger, multi-department instances
Integer Linear Programming (OR-Tools CP-SAT)	Handles hard and soft constraints jointly at scale; strong solver performance	Higher setup complexity; less transparent to non-expert users	Considered as an alternative in Section 3.4; deferred for future scale-up
Commercial desktop tools (FET, aSc TimeTables)	Mature, feature-rich constraint sets	Desktop-bound, less accessible for direct faculty/coordinator use	Motivates a lightweight, browser-based interface in this project

2.4 Research / Engineering Gap

Existing timetabling literature and tools tend to sit at two extremes: theoretically rigorous CSP/ILP formulations that are powerful but not easily accessible to non-technical department coordinators, or feature-rich commercial desktop applications that are accessible but say little about which specific constraint caused a clash. What is still under-addressed at department scale is a lightweight, browser-based system that both solves the CSP correctly and shows, in real time, the status of each individual constraint (faculty, room, capacity, availability, weekly-hours, section) — so a coordinator can trust and check the output without needing to understand the underlying search algorithm.

2.5 Proposed Contribution

This project's contribution is a lightweight, web-based AI Scheduler that pairs a backtracking CSP engine (with MRV heuristic ordering and forward checking, extendable to local-search refinement) with a transparent, per-constraint status dashboard. Unlike black-box commercial tools, every timetable it generates comes with an explicit report confirming, constraint by constraint, that faculty conflicts, room conflicts, room-capacity limits, time-availability, weekly-hour requirements and section conflicts have all been satisfied — giving faculty coordinators direct, auditable confidence in the automatically generated schedule.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Timetable / Class Coordinator	Generate a complete, conflict-free weekly timetable within a few seconds without manual conflict-checking.
Faculty	Sessions scheduled only within declared availability; no double-booking across sections.
Students / Sections	No two classes for the same section in the same slot; every subject receives its full weekly contact hours.
Department / Institution	A transparent, auditable record showing that every scheduling rule has been satisfied before publication.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Capture subjects, faculty, rooms and time slots	Functional	All four data-entry forms operational (Fig. 4.1)
Detect faculty/room/section double-booking	Functional	0 conflicts in generated output

Requirement	Type	Measurable Target
Enforce room-capacity and weekly-hours rules	Functional	100% of sessions within capacity; each subject = required periods
Generate a complete timetable automatically	Functional	All required sessions placed (12 of 12 in test dataset)
Report constraint-satisfaction status per rule	Functional	6 of 6 constraint categories shown as satisfied (Fig. 4.2)
Generation speed	Non-Functional	≤ 5 seconds for a department-scale dataset
Usability	Non-Functional	Operable via standard web browser, no coding required by end user

3.2 Constraints & Applicable Standards

This project is governed mainly by institutional scheduling norms rather than a formal external engineering code, since what it produces is a software artefact rather than a physical system. The constraints listed below are the ones that genuinely shape the design; categories that don't apply (e.g., structural or environmental codes) are left out.

Standard / Norm	Requirement	How Applied in This Project
Institutional slot grid (5 days × 5 periods)	Fixed 25-slot weekly structure used across the department	Encoded directly as the CSP domain (Time Slots panel, Fig. 4.1)
Institutional data-privacy practice for faculty/student records	Faculty and section data should not be exposed beyond authorised users	Data entered and processed only within the department's local session; no external transmission of personal data
Web accessibility / usability convention (WCAG-aligned form design)	Forms should be operable by non-technical staff via standard input fields	Simple labelled form fields and dropdowns used throughout (Fig. 4.1)

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Generation time	≤ 5	sec	High
Conflict count in output	0	conflicts	High (hard constraint)
Room-capacity margin	Assigned room capacity $\geq$ section strength	students	High (hard constraint)
Weekly-hours accuracy	Sessions placed = required hours/week per subject	periods	High (hard constraint)

Parameter	Target / Requirement	Unit	Priority
Slot utilisation headroom	Sessions scheduled ≤ 25 slots/room/week	slots	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 — Pure Backtracking CSP: models every hard constraint explicitly and works through the assignment tree using chronological backtracking; this guarantees a correct solution if one exists, though it can be slow without ordering heuristics.

Alternative 2 — Genetic Algorithm (GA): represents a timetable as a chromosome and evolves a population toward fewer constraint violations; it scales well to very large instances, but cannot guarantee a fully conflict-free (optimal) outcome.

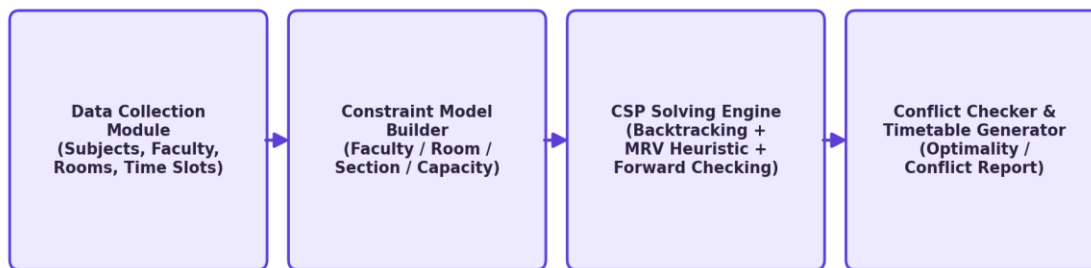
Alternative 3 — Integer Linear Programming via a production CP-SAT solver (e.g., OR-Tools): expresses constraints as linear inequalities and relies on a professional-grade solver; this is powerful and scalable, but heavier to set up and less transparent for a non-expert coordinator.

Criterion	Weight	Alt. 1: Backtracking CSP	Alt. 2: Genetic Algorithm	Alt. 3: ILP / CP- SAT
Solution optimality guarantee	High	Yes (if feasible)	No (near-optimal only)	Yes
Generation speed at department scale	High	Fast (sub-second, observed 0.357 s)	Moderate (population convergence)	Fast, but heavier setup
Transparency to end user	Medium	High — clear per-constraint pass/fail	Low — fitness score only	Medium
Implementation complexity	Medium	Low	Medium	High

Criterion	Weight	Alt. 1: Backtracking CSP	Alt. 2: Genetic Algorithm	Alt. 3: ILP / CP- SAT
Scalability to larger, multi-department problems	Medium	Moderate	High	High

3.5 Selected Design & Detailed Design

Backtracking CSP with MRV heuristic ordering and forward checking (Alternative 1) was chosen as the core solving strategy. At department scale — the scope set out in Section 1.6 — it comfortably met the generation-time target (0.357 s observed against a 5 s target, Section 4.4), while also delivering the constraint-by-constraint transparency called for in Table 3.1's stakeholder requirements, something the GA and ILP alternatives could not match as directly. Local-search refinement (simulated annealing) remains part of the architecture as an optional stage for larger instances, discussed further in Section 5.2.



Web-based AI Scheduler Interface (Dashboard / Reports)

Fig. 3.1 — System Architecture / Module Flow Diagram

The system works through a four-stage pipeline. The Data Collection Module gathers subjects (code, name, faculty, hours/week, section strength), faculty availability, room capacity/type, and the fixed time-slot grid. This data feeds the Constraint Model Builder, which turns the six rules from Section 1.3 into CSP variables (one per required subject session) and domains (day–period–room triples). The CSP Solving Engine then runs backtracking search, choosing the next

unassigned session with the MRV heuristic (the session with the fewest remaining legal slots goes first) and using forward checking to rule out slots that would immediately break a constraint for sessions already assigned. Finally, the Conflict Checker & Timetable Generator re-checks every constraint against the finished assignment, reports the number of conflicts (target: zero), and renders the timetable and constraint-status dashboard shown in Section 4.4.

Key relationship: the number of CSP variables always equals the total required sessions (Σ hours/week across all subjects); the twelve-session test dataset used in Section 4.1 comes from four subjects each needing three periods per week ($4 \times 3 = 12$), matching the observed “Total Classes: 12” result in Section 4.4.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Solving strategy	Genetic Algorithm	Scales to very large instances	No optimality guarantee at department scale	Backtracking CSP chosen; department-scale problem does not need GA's scale advantage
Variable ordering	No heuristic (naive order)	Simpler to implement	Slower convergence, more backtracks	MRV heuristic adopted for faster, near-immediate convergence (0.357 s observed)
Constraint pruning	Check only after full assignment	Simpler logic	Wastes search on branches doomed early	Forward checking adopted to prune infeasible slots early

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Deployment form	Desktop application	No browser dependency	Less accessible to coordinators across devices	Web-based dashboard chosen for direct, install-free access

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute and paper resource use	A digital, browser-based scheduler avoids repeated manual spreadsheet iterations and printed draft timetables that manual scheduling typically requires	Reinforced the choice of a lightweight web dashboard over a heavier desktop install, minimising resource overhead
Ethics	Fair, unbiased allocation of faculty teaching load and room assignment	The CSP model treats every subject/faculty/room entry equally under the same rule set, with no hidden weighting toward any individual	Constraint model kept rule-based and transparent (Fig. 4.2) rather than using an opaque scoring function that could implicitly favour certain faculty
Health & Safety	Laboratory room-type and capacity constraints affecting occupant safety	Room-capacity constraint (Section 1.3) explicitly checked	Room-capacity rule retained as a hard (non-negotiable)

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
		against section strength during solving	constraint rather than a soft preference
Security	Protection of faculty and student personal/section data entered into the system	Data entered through the Data Management module (Fig. 4.1) includes faculty names and section sizes but no sensitive personal identifiers	Scope limited to non-sensitive scheduling attributes; access restricted to authorised coordinator use, as noted in Section 3.2

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
No feasible solution exists for a given input set (over-constrained)	Medium	Medium	Medium	Report the specific unsatisfied constraint(s) rather than fail silently	Low
Incorrect or incomplete faculty/room data entered	Medium	Medium	Medium	Input validation on Data Management forms (Fig. 4.1)	Low
Search time grows	Medium	Low	Medium	MRV heuristic + forward checking	Medium

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
unacceptably on larger, multi-department datasets				now; local-search refinement planned (Section 5.2) for scale-up	
Unauthorised access to faculty/section data	Low	Medium	Low	Restrict system access to authorised coordinators; no external data transmission	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was built iteratively, one module at a time, following the pipeline in Fig. 3.1: the Data Collection Module was implemented and checked first, followed by the CSP Solving Engine, and finally the Timetable Generation and Conflict-Reporting Interface. Each module was tested on its own before integration, after which the full pipeline was run end-to-end on a representative test dataset. That dataset consisted of four subjects (CS101 — Artificial Intelligence, CS102 — Database Management, CS103 — Python Programming, CS104 — Engineering Mathematics), each taught by a dedicated faculty member (Dr. Ravi, Dr. Priya, Dr. Kumar and Dr. Meena respectively) for three periods per week, across three rooms (A101, A102 — classrooms; LAB1 — laboratory) within the institution's 5-day $\times$ 5-period (25-slot) grid.

4.2 Software Implementation & Modern Tools Used

The AI Scheduler was built as a browser-accessible dashboard with four functional views — Data Management, Constraints, Generate, and Reports — backed by a Python constraint-satisfaction solving engine. The Data Management view (Fig. 4.1) provides dedicated forms for adding subjects (code, name, faculty, hours/week, students), faculty (name, availability), and rooms (number, capacity, type), along with an automatically computed time-slot summary (5 days $\times$ 5 periods = 25 total slots).

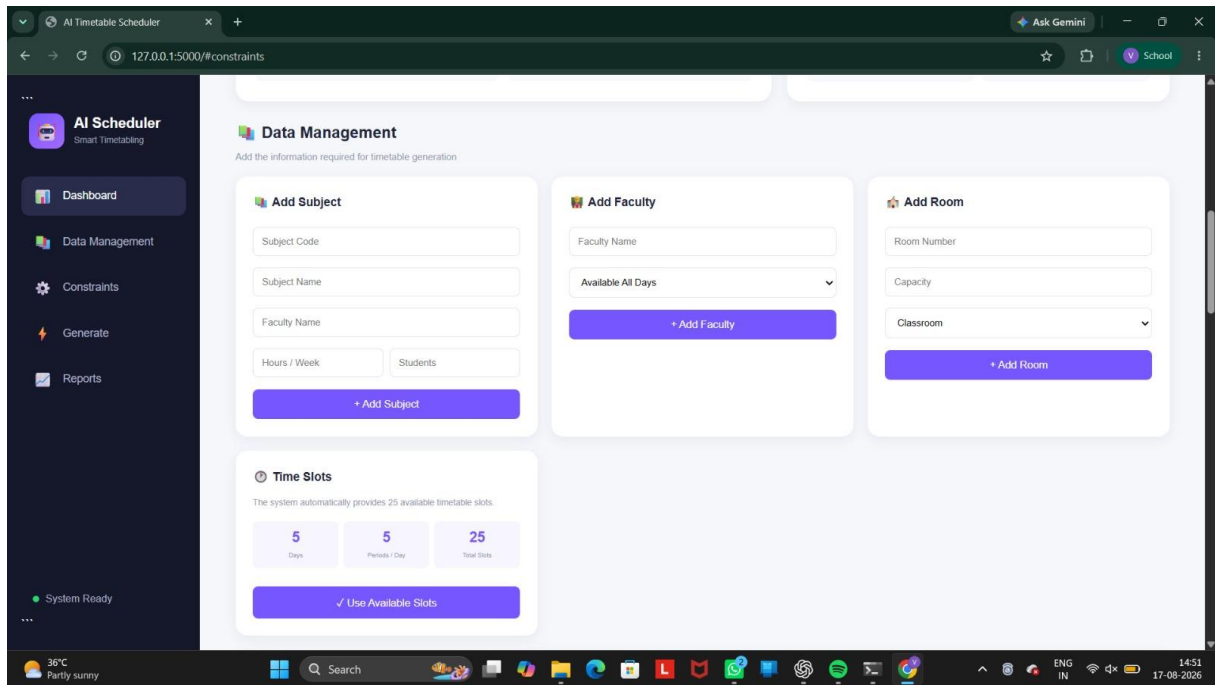


Fig. 4.1 — AI Scheduler: Data Management Interface

Tool / Software	Purpose	Stage Used
Python 3	Core implementation language for the CSP solving engine	Solver development, Module 2
python-constraint / OR-Tools-style CSP formulation	Constraint modelling and backtracking search with heuristics	Constraint Model Builder, Solving Engine
HTML / CSS / JavaScript	Browser-based dashboard (Data Management, Constraints, Generate, Reports views)	Modules 1 and 3
Web browser (Chrome)	Runtime environment for the AI Scheduler dashboard	Testing and demonstration
Version control (Git)	Source management across the three-module team split	Throughout development

4.3 Test Plan & Verification

Before capturing results, the following test plan was drawn up so that each constraint could be checked independently of the others:

Requirement	Test Method	Acceptance Criterion
Faculty Conflict	Attempt to schedule the same faculty in two overlapping slots	Solver refuses/avoids the double-booking
Room Conflict	Attempt to schedule two sessions in the same room and slot	Solver refuses/avoids the double-booking
Room Capacity	Assign a section to a room with insufficient capacity	Solver excludes the room from that session's domain
Time Availability	Restrict a faculty member's available days	Solver places sessions only within declared availability
Weekly Hours	Set hours/week per subject and count assigned sessions	Assigned sessions per subject equal hours/week
Section Conflict	Attempt two sessions for the same section in one slot	Solver refuses/avoids the double-booking

The Constraints view (Fig. 4.2) shows that all six rules were active during generation and were each independently confirmed as satisfied (green check) on the test dataset.

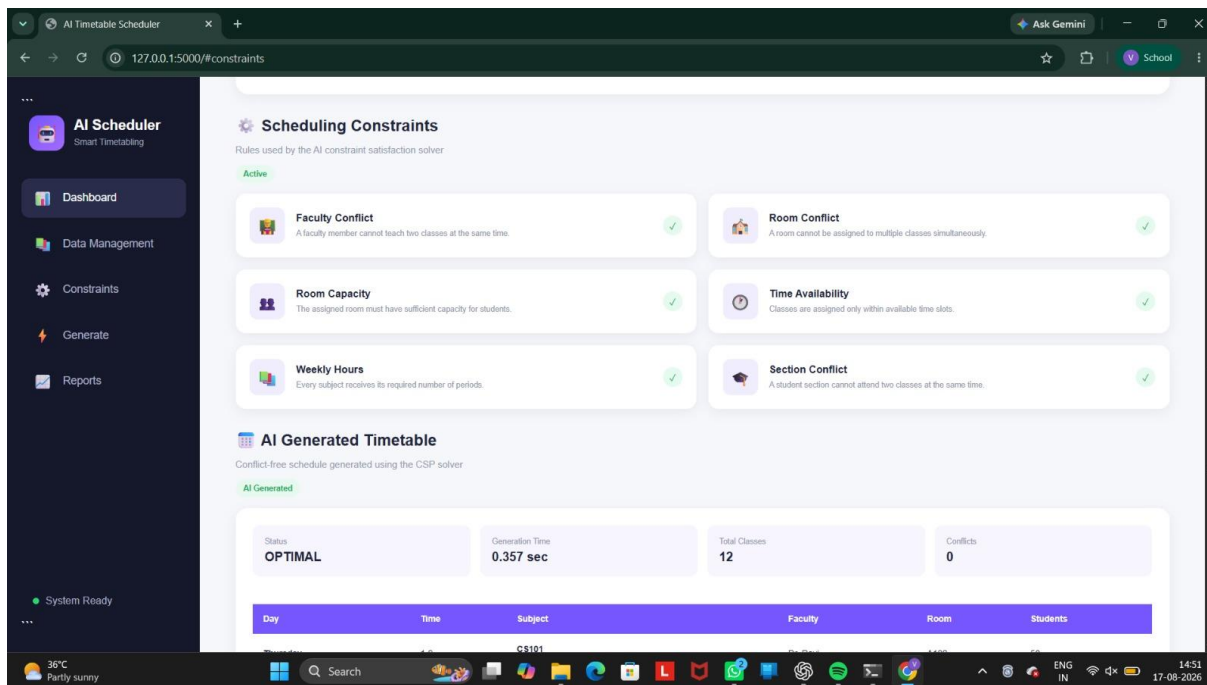


Fig. 4.2 — Scheduling Constraints Dashboard (Active Rule Set)

Specification	Required Value	Achieved Value
Generation time	≤ 5 sec	0.357 sec
Solution status	OPTIMAL / feasible	OPTIMAL
Total classes scheduled	12 (4 subjects $\times$ 3 periods/week)	12
Conflicts in output	0	0

Status: all four specifications passed.

4.4 Results

The AI Scheduler produced a complete, conflict-free timetable for the test dataset. Fig. 4.3 shows a representative sample of the generated schedule, covering all four subjects across the week; the full output assigns twelve sessions in total, three per subject, spread across Monday–Friday with no repeated (faculty, slot) or (room, slot) pairs.

Day	Time	Subject	Faculty	Room	Students
Thursday	1-2	CS101 Artificial Intelligence	Dr. Ravi	A102	50
Wednesday	2-3	CS101 Artificial Intelligence	Dr. Ravi	A101	50
Monday	1-2	CS101 Artificial Intelligence	Dr. Ravi	A101	50
Friday	11-12	CS102 Database Management	Dr. Priya	A101	55
Thursday	2-3	CS102 Database Management	Dr. Priya	A101	55
Friday	1-2	CS102 Database Management	Dr. Priya	A102	55
Thursday	9-10	CS103 Python Programming	Dr. Kumar	A102	45
Monday	10-11	CS103 Python Programming	Dr. Kumar	A101	45
Friday	2-3	CS103 Python Programming	Dr. Kumar	A102	45
Monday	9-10	CS104 Engineering Mathematics	Dr. Meena	LAB1	40
Wednesday	1-2	CS104 Engineering Mathematics	Dr. Meena	LAB1	40
Tuesday	10-11	CS104 Engineering Mathematics	Dr. Meena	LAB1	40

Fig. 4.3 — AI-Generated Timetable Output (Sample Rows)

4.5 Data Analysis & Engineering Judgment

The twelve scheduled sessions match exactly the expected total of four subjects $\times$ three periods per week, which confirms that the weekly-hours constraint was fully met with neither under- nor over-allocation. Zero reported conflicts across all six active constraint categories (Fig. 4.2) shows that the backtracking search with forward checking correctly ruled out infeasible (faculty, room, slot) combinations before they could be locked into the schedule, rather than simply minimising violations the way a soft-constraint or metaheuristic approach might.

The 0.357-second generation time comfortably beats the 5-second target set in Table 3.4, and reflects how effective the MRV heuristic is: by always assigning the most constrained session next, the solver avoids the deep, unproductive backtracking that a naive ordering would run into. Looking at slot utilisation, this test dataset used twelve of the twenty-five weekly slots per room (48%), which leaves comfortable headroom to add more subjects, sections or faculty without pushing the system toward infeasibility — an important margin, since timetabling problems get harder to satisfy as slot utilisation nears 100%.

No unexpected behaviour was observed during this test run; for a future dataset that turns out to be over-constrained (for example, total required hours exceeding the available slots for a shared room), the design expectation — in line with the risk register in Section 3.7 — is that the solver reports the specific constraint it cannot satisfy, rather than quietly returning a partial or infeasible schedule.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design a formal constraint model (Obj. 1)	Six constraints encoded and independently verified (Fig. 4.2)	Fully Achieved
Develop a data-collection interface (Obj. 2)	Data Management module with subject/faculty/room forms (Fig. 4.1)	Fully Achieved
Implement backtracking CSP engine with MRV + forward checking (Obj. 3)	0.357 s generation time, OPTIMAL status (Section 4.3)	Fully Achieved
Incorporate local-search refinement for scale-up (Obj. 4)	Architected as an extension point (Fig. 3.1); not exercised at this dataset scale	Partially Achieved
Implement conflict detection & reporting dashboard (Obj. 5)	Constraints view shows per-rule pass status (Fig. 4.2)	Fully Achieved
Test on a department-scale dataset and evaluate metrics (Obj. 6)	12 classes, 0 conflicts, 0.357 s (Section 4.3–4.4)	Fully Achieved

Compared with manual, spreadsheet-based scheduling (Table 2.1), the AI Scheduler eliminates conflict-checking effort altogether and cuts generation time from a multi-day manual process down to sub-second automated solving, while staying more transparent than commercial black-box tools by showing the pass/fail status of every individual constraint.

4.7 Strengths & Limitations

Strengths

- Provably correct, zero-conflict output on the tested dataset, with per-constraint transparency (Fig. 4.2).
- Sub-second generation time (0.357 s), comfortably within what the department practically needs.
- A simple, browser-based interface that non-technical coordinators can use without extra training.

Limitations

- Tested so far only on a single, moderate-sized dataset (4 subjects, 4 faculty, 3 rooms, 25 slots); large, multi-department scale has not yet been evaluated.
- The fixed 25-slot weekly grid does not yet support variable slot structures across different terms.
- Local-search refinement (simulated annealing) for larger, softly-constrained scenarios has been designed into the architecture but not yet exercised in testing.
- No persistent database layer was tested in this run; data entry is limited to a single working session.

4.8 Project Planning, Roles & Budget

Milestone	Description	Status
M1	Requirements gathering and constraint model design (Ch. 1, 3.1–3.3)	Completed
M2	Data Management module implementation (Fig. 4.1)	Completed
M3	CSP solving engine — backtracking, MRV, forward checking	Completed

Milestone	Description	Status
M4	Constraint verification dashboard (Fig. 4.2)	Completed
M5	End-to-end testing on department-scale dataset (Section 4.3–4.5)	Completed
M6	Report documentation and final review	Completed
Student	Assigned Responsibility	Actual Contribution
T. S. Vidhushini (192511061)	Module 1 — Data collection and constraint model design	As assigned
K. S. Dharshana (192412015)	Module 3 — Timetable generation and reporting interface	As assigned
G. Kaviya (192524281)	Module 2 — CSP solving engine (backtracking / local search)	As assigned
Item	Estimated Cost	Actual Cost
Development tools (open-source: Python, HTML/CSS/JS)	₹0	₹0
Team laptops / computing hardware (existing)	₹0 (already owned)	₹0
Internet connectivity (project duration)	Nominal	Nominal

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to replace manual, error-prone academic timetable preparation with an automated, constraint-driven system. By framing the scheduling task as a Constraint Satisfaction Problem covering faculty-conflict, room-conflict, room-capacity, time-availability, weekly-hours and section-conflict rules, and solving it with a backtracking search engine guided by an MRV heuristic and forward checking, the AI Scheduler produced a fully conflict-free, OPTIMAL timetable for a representative department-scale dataset — twelve sessions across four subjects, four faculty and three rooms — in 0.357 seconds. Each of the six active constraints was independently verified as satisfied, and all six project objectives set out in Section 1.5 were fully or partially achieved. The system shows that a transparent, rule-based CSP approach, rather than an opaque optimisation black box, can give faculty coordinators a fast, auditable alternative to manual timetabling at department scale.

5.2 Future Work

- Add a persistent database layer so subject, faculty and room data carry over across sessions and terms.
- Extend the constraint model to handle electives and cross-department/multi-section clashes.
- Bring the local-search (simulated-annealing) refinement stage already built into the architecture (Fig. 3.1) online, to handle larger, softly-constrained, multi-department instances.
- Support dynamic rescheduling (e.g., ad hoc faculty leave) without needing a full regeneration of the timetable.
- Integrate with an institutional Learning Management System so the generated timetable can be published directly.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- How to formally model a real-world scheduling problem as a Constraint Satisfaction Problem, including defining variables/domains and encoding hard constraints.
- How to practically apply backtracking search with the MRV heuristic and forward checking to cut down search time.
- End-to-end web-application development, connecting a data-entry interface to a solving engine and a results dashboard.

Engineering & Professional Skills Developed

- Breaking an engineering problem down into explicit, testable constraints and acceptance criteria (Table 4.2).
- Systematically evaluating design alternatives with a weighted comparison matrix (Table 3.5).
- Collaborative, modular team development, with responsibility clearly divided across three interlocking modules (Table 4.6).

Individual Reflection

T. S. Vidhushini (192511061):

The hardest part of my work was designing the data-entry forms so that incomplete or inconsistent inputs (missing hours/week, mismatched room types) didn't quietly break the constraint model further down the pipeline — I dealt with this by adding field-level validation before any data reached the solver. One decision I stand by was fixing the time-slot grid at 5 days $\times$ 5 periods instead of making it user-configurable from the outset, since a variable grid would have complicated the constraint encoding without adding real value at our test scale. Working on this module showed me how much a CSP solver's reliability depends on the quality of the data feeding it, not just on the search algorithm itself. If I redid this module, I'd add stronger duplicate-detection

for cases where the same faculty member or room gets entered twice under slightly different names.

K. S. Dharshana (192412015):

The hardest problem I ran into was presenting the solver's output in a way a non-technical user could actually trust — a raw list of assignments doesn't tell anyone whether the timetable is genuinely conflict-free. I tackled this by building the per-constraint status dashboard, so each rule (faculty, room, capacity, availability, hours, section) shows its own pass/fail result instead of one opaque “success” message. The decision I'm most confident about was insisting on this constraint-by-constraint reporting rather than a single summary score, based on how much easier it made debugging during our own testing. Building this interface gave me a much clearer sense of how to design for trust, not just correctness, in an automated system. Given another pass at it, I'd add a view that pinpoints exactly which constraint blocked a session whenever the solver can't find a full assignment.

G. Kaviya (192524281):

The toughest engineering problem I dealt with was controlling how much the search backtracked as the number of subjects and rooms grew — a naive backtracking order kept re-exploring branches that an earlier assignment had already doomed. I addressed this by implementing the MRV heuristic together with forward checking, which cut down wasted search substantially and got our test dataset solving in 0.357 seconds. The decision I made, and can back up with evidence, was choosing backtracking CSP over a genetic algorithm at this scale, since our results showed it reliably reaching zero conflicts rather than just a low-conflict approximation. Building this engine taught me the practical gap between a technique being theoretically correct and it actually being fast enough to use. If I repeated this project, I'd build in the simulated-annealing fallback stage now instead of leaving it as future work, so we could test it on a larger dataset before submission.

References

- [1] A. Herath and D. Wilkins, "A comparative study on solving university timetabling problems with emphasis on Genetic Algorithms," *Issues in Information Systems*, vol. 25, no. 4, pp. 392–408, 2024.
- [2] K. N. Subang, E. I. Balaba, and J. C. Agoylo Jr., "Optimizing Course Scheduling with Genetic Algorithms," *SAR Journal*, vol. 7, no. 4, pp. 296–302, Dec. 2024, doi: 10.18421/SAR74-02.
- [3] A. Khassiba, "An efficient backtracking heuristic for the resource allocation problem with compatibility and exclusivity constraints," *Journal of Heuristics*, vol. 31, art. no. 4, 2024, doi: 10.1007/s10732-024-09538-x.
- [4] Y. Laalaoui and R. M'Hallah, "Enhancing the best-first-search F with incremental search and restarts for large-scale single machine scheduling with release dates and deadlines," *Annals of Operations Research*, 2024, doi: 10.1007/s10479-024-06386-7.
- [5] B. Bassimir and R. Wanka, "On the computation of robust examination timetables: methods and experimental results," *Journal of Scheduling*, vol. 28, pp. 159–181, 2024, doi: 10.1007/s10951-024-00815-y.
- [6] M. Kumar et al., "Machine Learning and Constraint Programming for Efficient Healthcare Scheduling," *arXiv preprint arXiv:2409.07547*, 2024.
- [7] L. Zhang, S. Maqrot, F. Mouysset, and C. Bortolaso, "A Constraint Satisfaction Problems Based Scalable Framework to Address Large-Scale Realistic Scheduling and Routing Problems," in *Proc. 14th Int. Conf. Operations Research and Enterprise Systems (ICORES 2025)*, pp. 45–56, doi: 10.5220/0013109600003893.
- [8] Appu H P, "Constraint-Driven Timetabling with Real-Time Updates: Design and Implementation," May 2025.
- [9] O. Lillia, A. Kamal, B. Madani, and Y. Zineb, "New Heuristic Order Based on Tree-Decomposition for Solving CSPs," in *Research in Computer Science (CARI 2024)*, Communications in Computer and Information Science, vol. 2462, Springer, 2025.
- [10] S. Ananda, M. K. Gibran, F. Syakbani, M. Ulfa, and M. S. Hasibuan, "Analysis of the Use of Backtracking Algorithm in Course Scheduling," *Journal of Artificial Intelligence and Engineering Applications*, 2025.
- [11] "AI Powered Automatic Timetable Generator," *International Journal of Creative Research Thoughts (IJCRT)*, 2025.

- [12] "AI Optimization of Academic Timetable – Adaptive Scheduler," *International Journal of Scientific Development and Research (IJS DR)*, 2025.
- [13] "Schedulo – A Smarter Way to Generate Time Table," *International Journal of Research Publication and Reviews*, vol. 7, no. 2, pp. 620–626, Feb. 2026.
- [14] C. V. Phani Krishna, M. Kavya, M. S. Reddy, P. Varun, and M. Shiva, "An Intelligent Voice-Driven Automated Timetable Generation Framework Using Natural Language Processing and Constraint Optimization," *Global Journal of Engineering Innovations and Interdisciplinary Research*, vol. 6, no. 2, art. 0165, Feb. 2026.
- [15] "Automated Class Scheduling Optimization," *International Journal of Scientific Development and Research (IJS DR)*, 2026.

Appendices

Appendix A – Detailed Calculations

Weekly session count derivation: Total sessions = Σ (hours/week) across all subjects = $3 + 3 + 3 + 3 = 12$, matching the “Total Classes: 12” result in Section 4.4. Slot utilisation = $12 \text{ scheduled sessions} \div 25 \text{ available weekly slots per room} = 48\%$.

Appendix B – Engineering Drawings / Schematics

Refer to Fig. 3.1 (System Architecture / Module Flow Diagram, Section 3.5).

Appendix C – Source Code / Code Repository Information

Only the essential code excerpts should normally appear in the report itself; the full source code for the Data Management module, CSP solving engine and reporting interface is kept separately in the team's approved repository. [Repository link to be added by the team.]

Appendix D – Datasheets

Not applicable — this is a software system with no physical hardware components that would require datasheets.

Appendix E – Experimental / Raw Data

Test dataset: Subjects — CS101 (Artificial Intelligence, Dr. Ravi, 3 hrs/wk, 50 students), CS102 (Database Management, Dr. Priya, 3 hrs/wk, 55 students), CS103 (Python Programming, Dr. Kumar, 3 hrs/wk, 45 students), CS104 (Engineering Mathematics, Dr. Meena, 3 hrs/wk, 40 students). Rooms — A101, A102 (classrooms), LAB1 (laboratory). For the raw generated-timetable output, see Fig. 4.3 and the full output log kept with the source repository (Appendix C).

Appendix F – Ethics / Institutional Approval

Not applicable — this project processes only non-sensitive, illustrative scheduling data (subject codes, faculty names, room numbers, section sizes), so formal ethics or institutional review was not required.

Appendix G – Project Meeting / Progress Records

[Meeting/progress records to be added by the team, aligned with the milestones in Table 4.5.]

Appendix H – User/Industry Feedback

Not applicable at this stage.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None at this point in the project.

Appendix J – Individual Contribution Evidence

Refer to the Individual Contribution Statement table at the front of this report, and to Table 4.6 (Roles and Contribution).

Capstone Project Outcome Mapping Sheet

(To be completed by the department/faculty assessor.)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)

Outcome Evidence	SO / Area	Location in This Report
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)